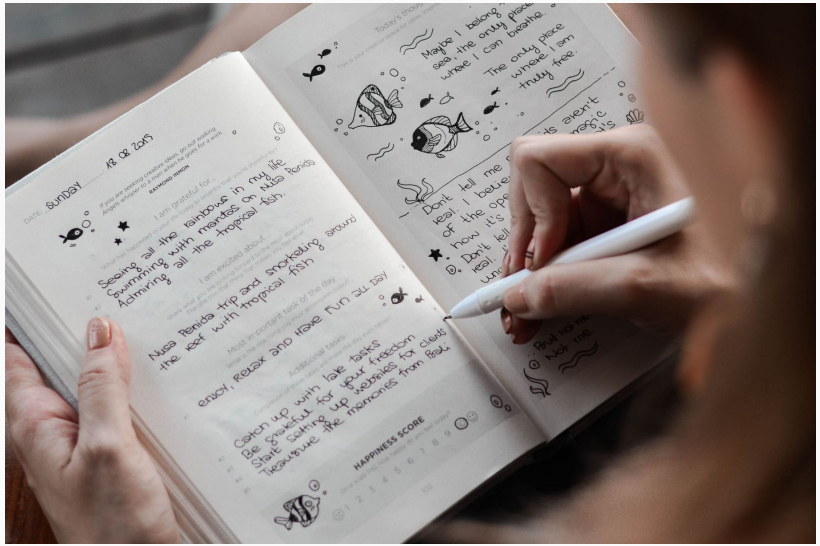
ReWind Check-in 2

April 2026

Pardesh Dhakal, Angel Manson, & Cheyenne Mowatt

App Overview



- Many people want to journal consistently, yet most existing journaling tools focus only on writing rather than reflection.
- Users often capture thoughts in the moment but rarely return to analyze patterns, growth, or emotional change over time.
    - Journaling becomes a collection of isolated entries rather than a meaningful narrative.
- This application aims to transform journaling from a sporadic habit into a long-term practice supported by AI.

MVVM

Model

- Data layer: Room database entities (JournalEntry, Folder, etc. – their schemas) and the JournalRepository.
- The Repository provides an abstraction to the ViewModel.

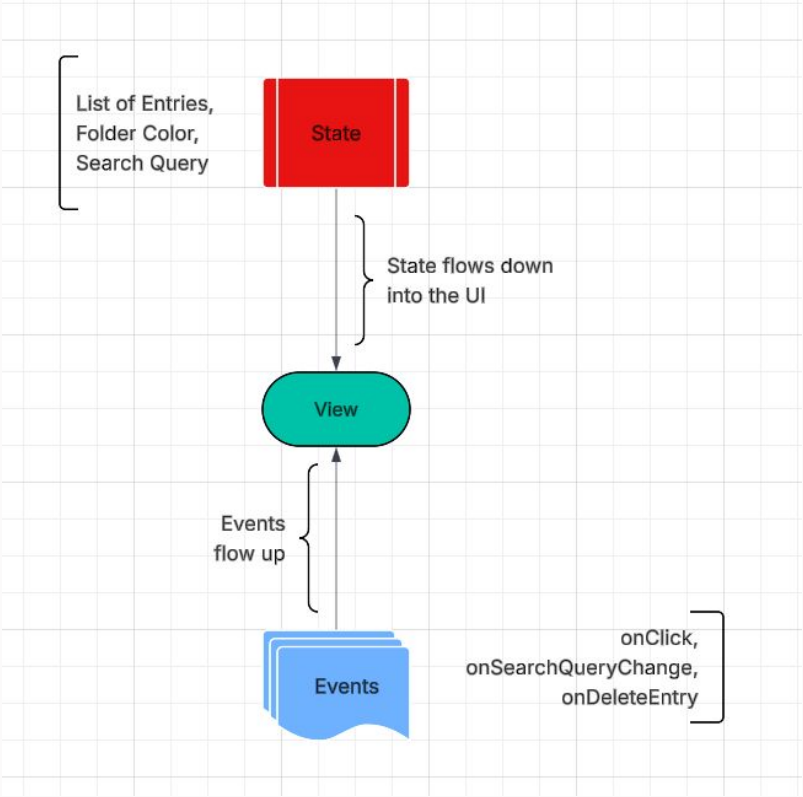
ViewModel

- Logic layer: (JournalViewModel.kt) holds the State of the UI and handles user actions.
- Coordinates with the UI and the repository
- In ReWind:
    - It stores the current search query and applies search filtering
    - It calls the repository to load and modify data (entries, folders, etc.) and gives to UI as StateFlows.

View

- UI layer: Describes how the application is composed
- UI is split up into screens: HomeScreen, NewEntryScreen, FolderScreen, TimelineScreen, etc.
- Reusable components are kept in the Components.kt file such as Search Bar

Unidirectional Flow



- ViewModel provides a StateFlow. Compose collects this and draws the UI
- Events/Actions are sent to the ViewModel, which updates the underlying data
- Data driven and single source of truth ensured

Dependencies - Quick Overview

Category	Key Dependency	Purpose
UI	Jetpack Compose	Declarative UI & Material 3 Design
Database	Room	Structured local storage with Flow support
Network	Retrofit	Fetching external data (e.g. Affirmations api)
Logic	Coroutines	Thread management & reactive data streams

Database Architecture (Room)

- **Folder Entity**: Stores metadata including a unique ID, Name, Description, and a hex Color code used for dynamic UI theming across the app.
- **Journal Entry Entity**: Captures user data (Title, Body), an automatic Timestamp, and optional location data (Latitude/Longitude).
- **One-to-Many Relationship**: Folders and Entries are linked via a nullable folderId. This allows entries to belong to a specific folder or remain "General".

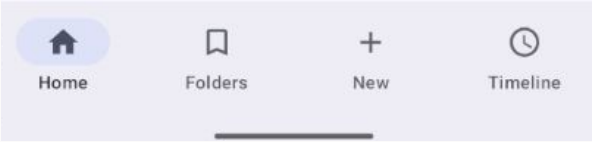
```
@Entity(tableName = "folders")
data class Folder(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
    val description: String = "",
    val color: Int = 0xFF6200EE.toInt(), // Default color (Purple)
    val timestamp: Long = System.currentTimeMillis()
)

15 Usages
@Entity(
    tableName = "journal_entries",
    foreignKeys = [
        ForeignKey(
            entity = Folder::class,
            parentColumns = ["id"],
            childColumns = ["folderId"],
            onDelete = ForeignKey.SET_NULL // Entries will survive their folder's delete
        )
    ],
    indices = [Index(value = ["folderId"])]
)
data class JournalEntry(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val folderId: Long? = null,
    val title: String,
    val body: String,
    val timestamp: Long = System.currentTimeMillis(),
    val latitude: Double? = null,
    val longitude: Double? = null
)
```

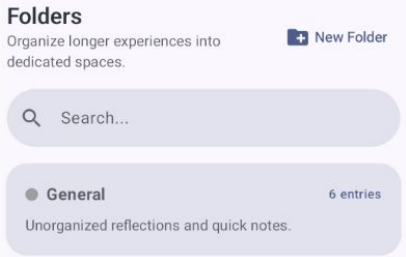
Navigation Flow

Structure

- The app uses a hierarchical navigation structure managed in MainActivity.
- Root Screens & Secondary Screens (Nested) based on user activity
- We have ‘selectedTab’ logic that determines which screen is active



- PersistenceBottom nav-bar for user switching between screens



- Secondary Screens from the folder screen -> New Folder or -> Tap Folder -> Folder Detail View

More rigorous demonstration will be included in the demo!

Debugging and Testing

- Used Compose Previews to iterate on cards and dialogs without launching the full app.
- Used Room In-Memory Database tests to ensure relationships and search queries work correctly.
- "Edge case" testing (e.g., making sure that deleting a folder keeps all the entries).
- Used Database inspector and logcat for debugging

What's New?

Key Features

- The complete base level journaling experience is complete
- Integrated daily affirmations API
- Auth code set up but currently being debugged

Local Persistence

- Data is stored locally in an encrypted form
- Users need to create an account

Sensors

- Use location sensors to track location

Scope and Descope

Progress

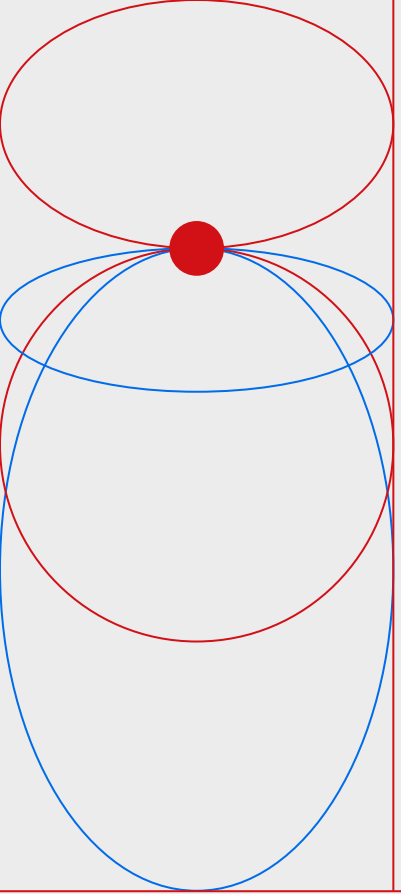
- Real-time filtering of both folders and entries on a single screen (search bar)
- Dynamic Theming: Folder color selection that affects to recent entry indicators.
- Folders: Implementation of the "General" folder for unorganized thoughts and custom user folders

Descoped

- No longer pursuing the Insights panel (from Version 1.0)
    - All focus shifted to our two timelines

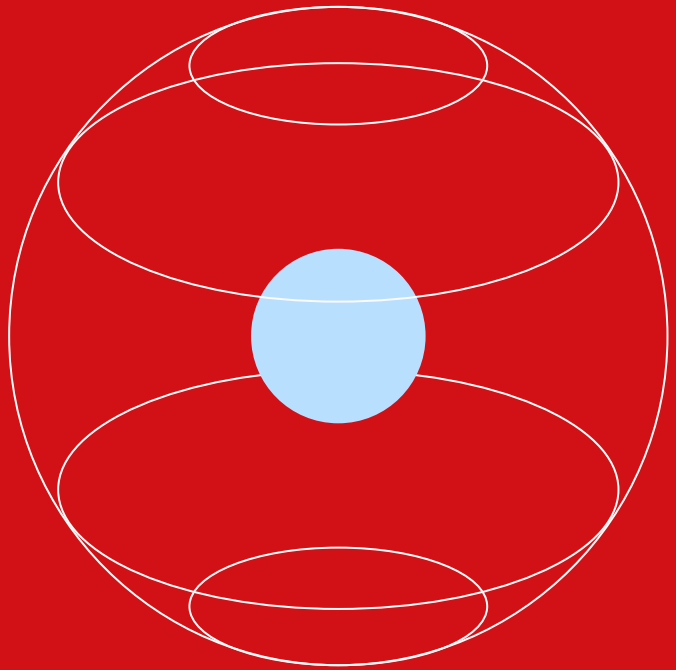
To Do

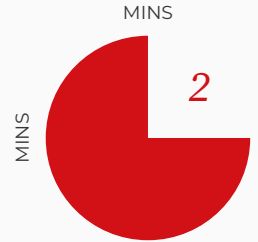
- Cloud Sync: Firebase integration for cross-device access. (debugging)
- Security: local database encryption via SQLCipher.
- AI Insights: Offloading sentiment analysis to the cloud to generate weekly reflection summaries
    - Currently testing AI API tools such ChatGBT on sample data
    - Experimenting with ML APIs such as IBM Watson NLU and Twinword for emotion detection

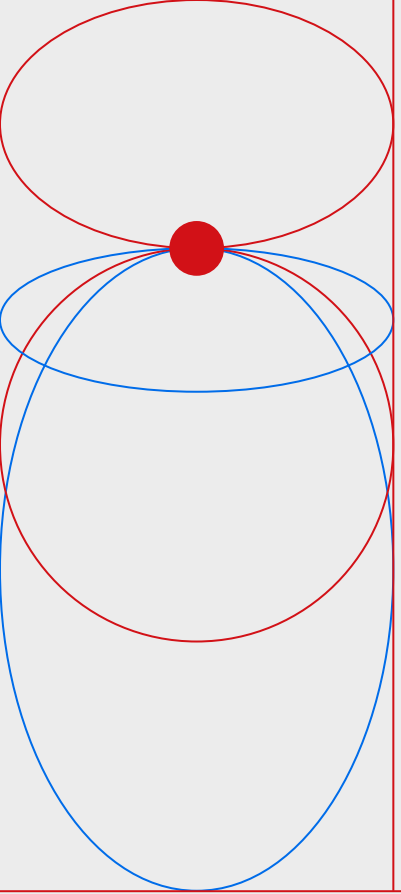


Demo

Any Questions?







Thank you